

UCS503P - Software Engineering

AI-Enabled Smart Library Management and Resource Recommendation System

Prototype Report

Submitted by:

1024160010 Arnav Agarwal
1024160135 Aradhya Goyal
1024160025 Amanjot Kaur Sidhu

BE Third Year, CSE

Submitted to:

Dr. Jeelani Asif

Prototype Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	7
1.1	Background and Context	7
1.2	Problem Statement	7
1.3	Project Objectives	8
2	Methodology and Design	9
2.1	Development Methodology	9
2.2	System Architecture	9
2.2.1	High-Level Design	9
2.2.2	Technical Stack	10
2.3	System Design and Diagrams	10
2.3.1	Use Case Diagram	10
2.3.2	Data Flow Diagram	11
2.3.3	Entity Relationship Diagram	14
2.3.4	Activity and Swimlane Diagram	15
2.4	Database Design	16
2.5	Library Intelligence Agent	17
2.6	Record Onboarding Agent	17
2.7	Library Monitoring Agent	18
2.8	AI Safety and Control	18
2.9	CI/CD and Deployment	19
3	Prototype Implementation and Evaluation	21
3.1	Prototype Implementation	21
3.2	Core Library Workflow	21
3.3	Intelligent Resource Discovery Workflow	22
3.4	Record Onboarding Workflow	22
3.5	Monitoring and Notification Workflow	23
3.6	Testing Methodology	23
3.6.1	Unit Testing	23
3.6.2	Integration Testing	24
3.6.3	Performance Testing	24
3.6.4	Functional Testing	24
3.7	Evaluation Criteria	24
3.7.1	Primary Metrics	25
3.7.2	Secondary Metrics	25
3.8	Validation Strategy	25
3.9	Analysis	26

4	Scalability and Deployment	27
4.1	Scalability	27
4.1.1	Logical Scalability	27
4.1.2	Operational Deployability	27
4.2	Engine Availability	27
4.3	Deployment Strategy	28
4.4	Risks and Mitigations	28
5	Conclusion	29
5.1	Summary of Work	29
5.2	Key Findings	29
5.3	Future Work and Recommendations	29
5.4	Final Remarks	30

List of Figures

2.1	Use Case Diagram of the Smart Library Management System.	11
2.2	Level 0 Context Data Flow Diagram.	12
2.3	Level 1 Data Flow Diagram showing the major system subsystems. .	13
2.4	Level 2 Data Flow Diagram for Intelligent Resource Search.	14
2.5	Entity Relationship Diagram of the Smart Library Management System.	15
2.6	Activity and Swimlane Diagram for Intelligent Resource Discovery and Borrowing.	16

Introduction

1.1 Background and Context

University libraries manage large collections of books and other learning resources while handling student records, issue and return transactions, overdue items, fines, and resource availability. Conventional library management systems primarily provide storage, search, and CRUD-based operations, leaving several administrative activities manual and reactive.

The proposed Smart Library Management System combines conventional library workflows with a lightweight multi-agent artificial intelligence layer. The system is designed to assist students with resource discovery and recommendation while helping administrators with record onboarding, overdue monitoring, notifications, and resource-demand analysis.

The project uses artificial intelligence as an enabling technology rather than as a research objective. The primary focus remains dependable software engineering, workflow automation, system integration, measurable improvement, and practical deployment.

1.2 Problem Statement

University libraries handle a large number of resource records, student records, borrowing transactions, and overdue items. Although conventional library systems can store and retrieve these records, several operational problems remain.

- **Manual resource discovery:** Students may need to navigate through multiple search and filtering options to determine whether a suitable resource is available.
- **Manual record onboarding:** Importing large numbers of student or member records can require repetitive validation and manual duplicate checking.
- **Reactive overdue management:** Overdue resources may only be identified after staff manually inspect issue and due-date records.
- **Limited demand visibility:** Library administrators may not have an immediate way to identify frequently issued resources with insufficient available copies.

- **Data without assistance:** A conventional database stores library information but does not directly assist users in converting that information into actionable answers or recommendations.

Why this matters now: A university library serves a large student population with continuously changing resource requirements. Improving resource discovery, record processing, overdue monitoring, and demand visibility can reduce repetitive administrative effort and improve the usability of existing library resources.

1.3 Project Objectives

The prototype is designed to achieve the following objectives:

1. Develop a functional web-based Smart Library Management System.
2. Provide resource and student/member management.
3. Support resource search and real-time availability display.
4. Implement issue, return, due-date, and fine workflows.
5. Provide natural-language resource discovery using the Library Intelligence Agent.
6. Provide bulk record validation and duplicate detection using the Record Onboarding Agent.
7. Provide automated overdue monitoring and notification support using the Library Monitoring Agent.
8. Analyse resource demand and availability for administrative decision support.
9. Maintain controlled interaction between AI agents and the underlying database.
10. Support automated testing, continuous integration, and deployment.

Methodology and Design

2.1 Development Methodology

The project follows an incremental engineering approach based on rapid time-to-value and short feedback cycles.

The initial implementation focuses on establishing a usable minimum viable product containing authentication, resource management, resource search, issue and return workflows, and database integration.

AI-assisted functionality is introduced incrementally after the core library workflow is operational. Continuous integration is established early so that code changes can be checked through automated testing, linting, and build validation.

Each development iteration produces a demonstrable increment that can be tested independently and improved using feedback.

2.2 System Architecture

The system follows a modular three-tier web architecture integrated with a controlled AI agent layer.

2.2.1 High-Level Design

The major components of the system are:

1. **Student / Administrator Web Interface:** Provides interfaces for students and administrators to access library services and management functions.
2. **Backend Application and REST APIs:** Handles authentication, authorization, library workflows, resource management, student records, transactions, and communication with the AI agents.
3. **MySQL Database:** Stores library resources, student/member records, borrowing transactions, due dates, fines, and availability information.
4. **Controlled AI Agent Layer:** Provides the Library Intelligence Agent, Record Onboarding Agent, and Library Monitoring Agent through controlled application APIs and tools.

The architecture separates deterministic operations from AI-assisted interpretation and recommendation.

Critical operations such as issue and return transactions, availability calculation, due-date checking, and fine calculation are handled by the backend and database layers.

2.2.2 Technical Stack

The project uses mature technologies suitable for rapid engineering and deployment.

- **Frontend:** React.js / Web Client
- **Backend:** REST-based application service
- **Database:** MySQL
- **AI Layer:** Controlled AI agents with structured outputs
- **Communication:** REST APIs and structured JSON
- **Testing:** Automated unit and integration testing
- **CI/CD:** Automated build, test, and deployment pipeline
- **Deployment:** Containerized or cloud-hosted application

2.3 System Design and Diagrams

2.3.1 Use Case Diagram

The Use Case Diagram defines the interactions between the primary human actors, Students and Librarian, and the three specialized AI agents.

The system uses the following final agent terminology:

- Library Intelligence Agent
- Record Onboarding Agent
- Library Monitoring Agent

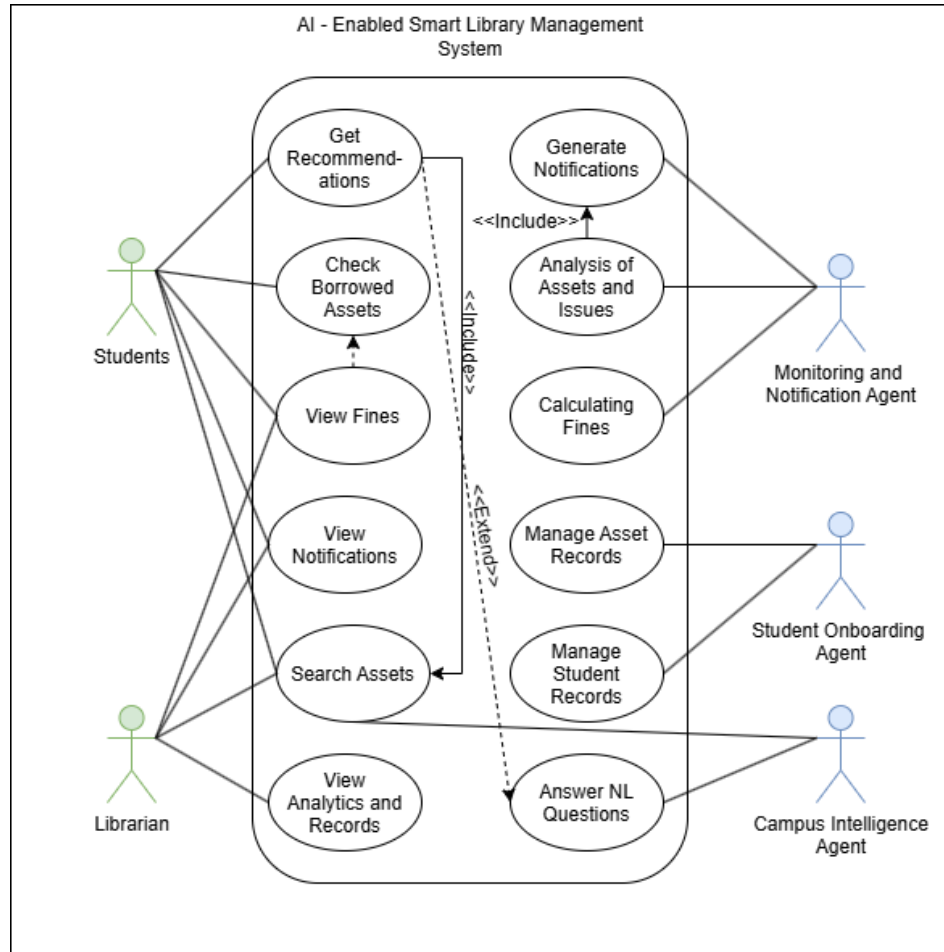


Figure 2.1: Use Case Diagram of the Smart Library Management System.

2.3.2 Data Flow Diagram

The Data Flow Diagrams describe the movement of information between students, librarians, the Smart Library Management System, the database, and the controlled AI agent layer.

Level 0 – Context Diagram

The Level 0 Context Diagram presents the overall system boundary and the major information exchanges between users, AI agents, and the Smart Library Management System.

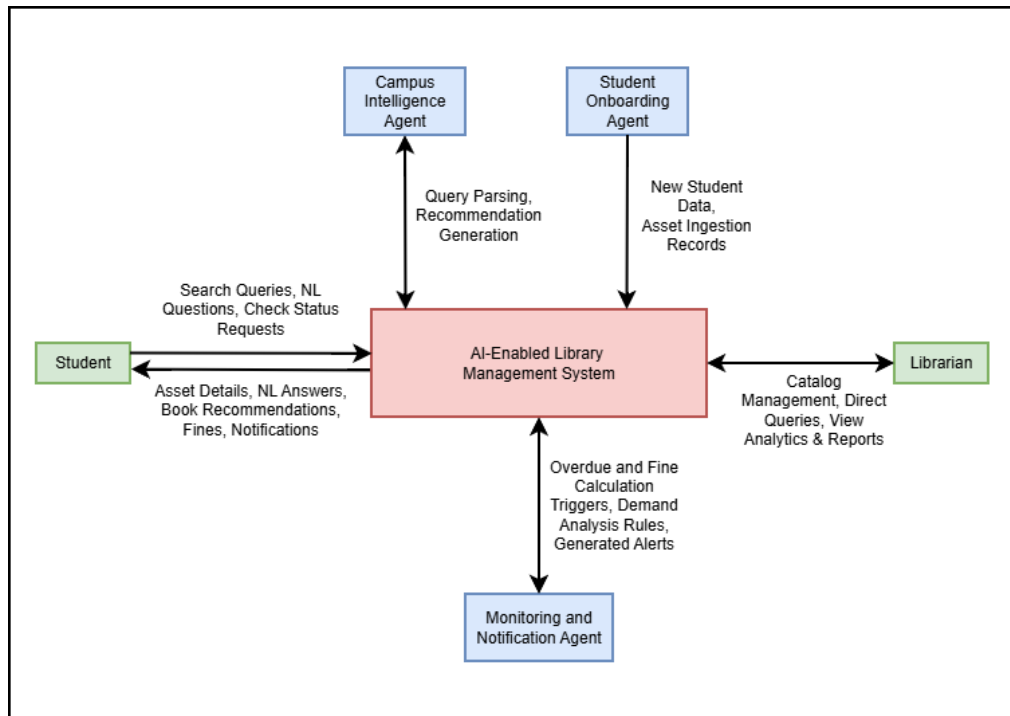


Figure 2.2: Level 0 Context Data Flow Diagram.

Level 1 – System Subsystems

The Level 1 DFD decomposes the Smart Library Management System into its major functional subsystems, including intelligent search, library records and resource management, circulation and fine tracking, and monitoring and notification.

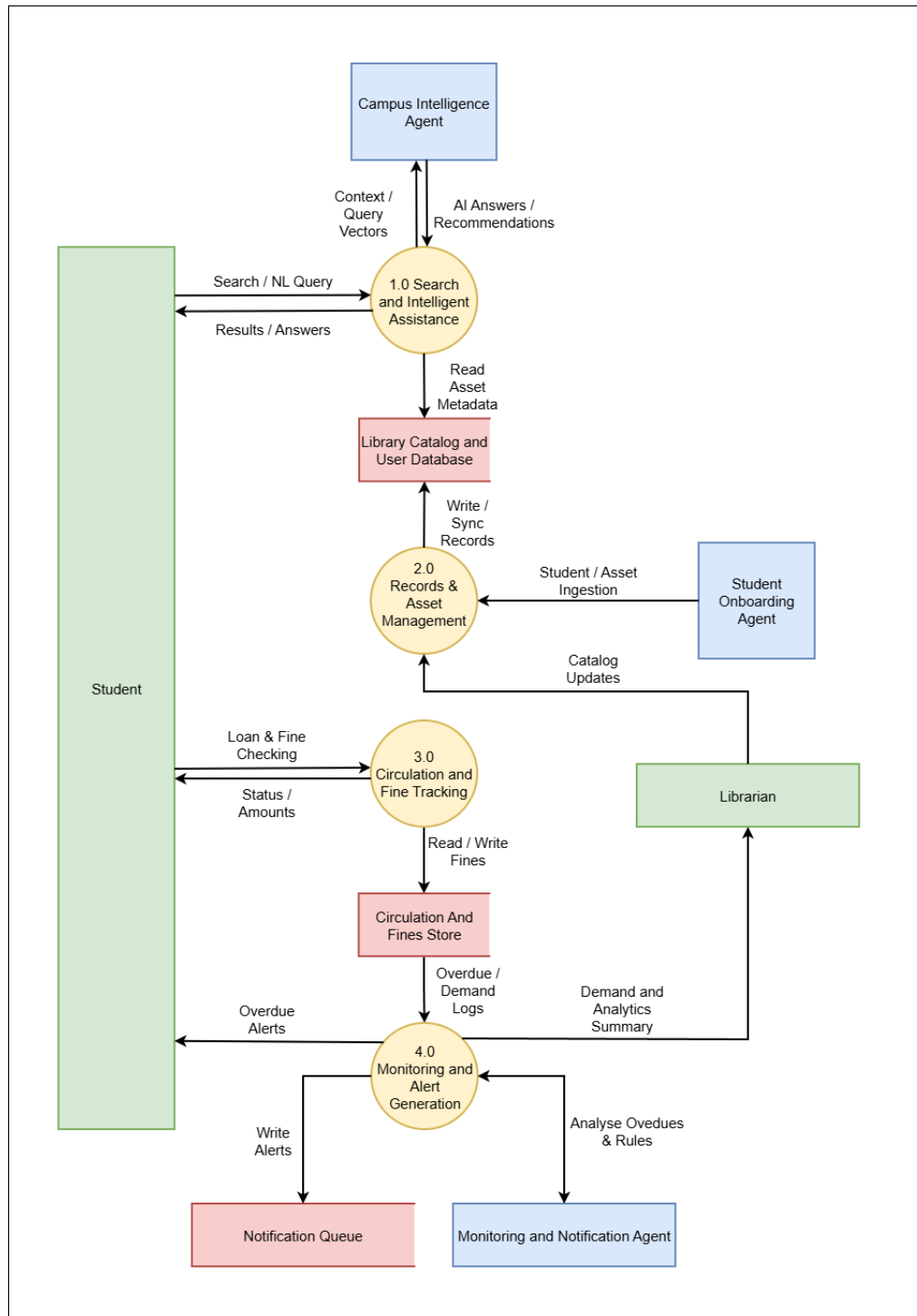


Figure 2.3: Level 1 Data Flow Diagram showing the major system subsystems.

Level 2 – Intelligent Search Process

The Level 2 DFD provides a detailed representation of the intelligent resource-search workflow, including natural-language query processing, catalogue retrieval, availability checking, and recommendation generation.

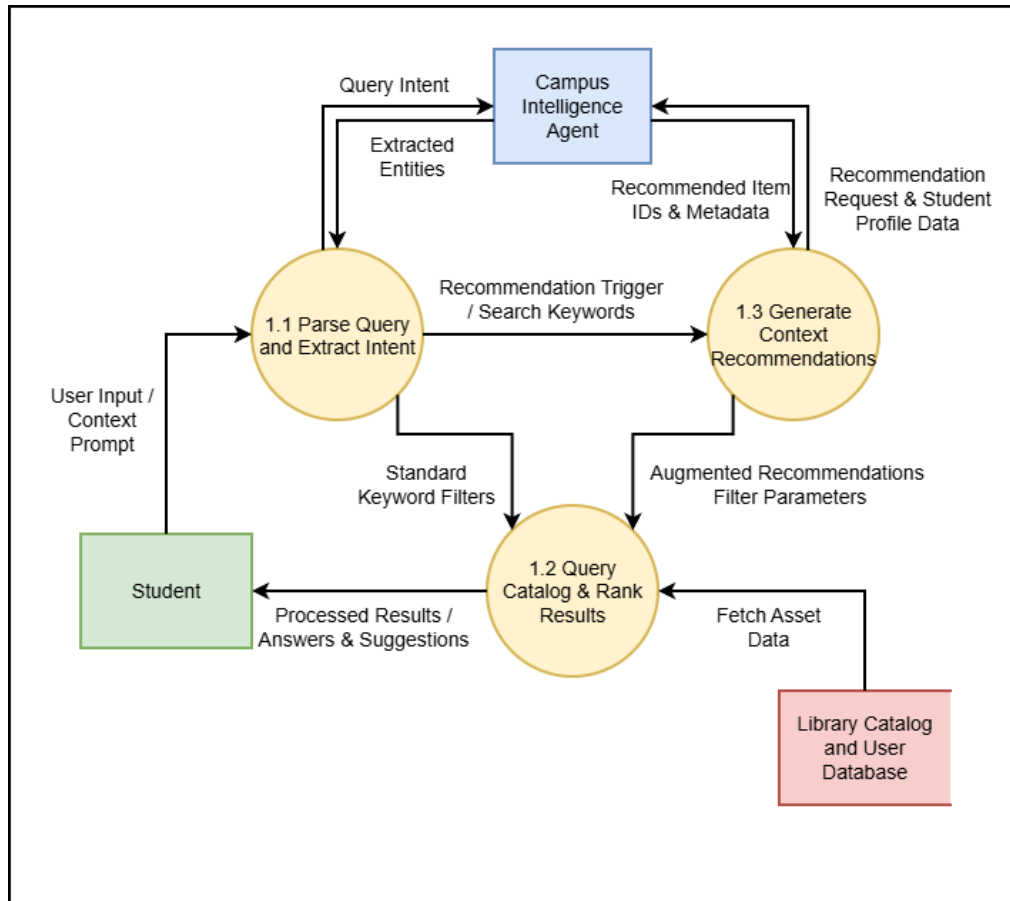


Figure 2.4: Level 2 Data Flow Diagram for Intelligent Resource Search.

2.3.3 Entity Relationship Diagram

The Entity Relationship Diagram represents the principal database entities and their relationships.

The major entities include:

- STUDENTS
- EMPLOYEE
- ASSETS

- BOOKS
- ISSUING_DETAILS
- FINES
- NOTIFICATION_QUEUE
- SEARCH_DEMAND_LOGS

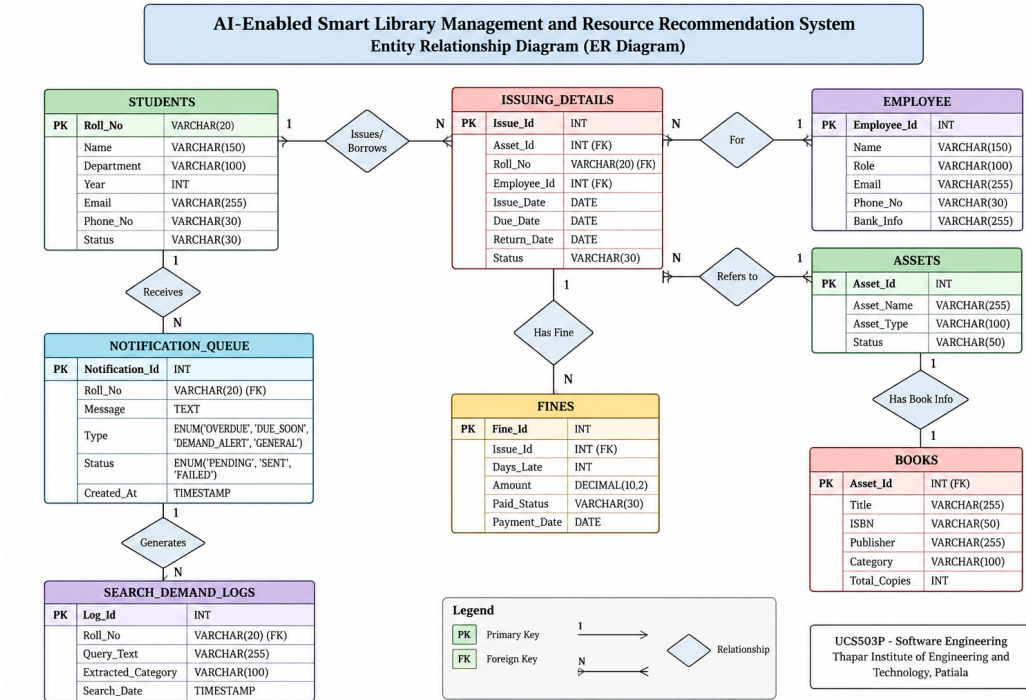


Figure 2.5: Entity Relationship Diagram of the Smart Library Management System.

2.3.4 Activity and Swimlane Diagram

The Activity and Swimlane Diagram illustrates the interaction between the student, web interface, backend application, Library Intelligence Agent, MySQL database, and librarian/notification layer during intelligent resource discovery and borrowing.

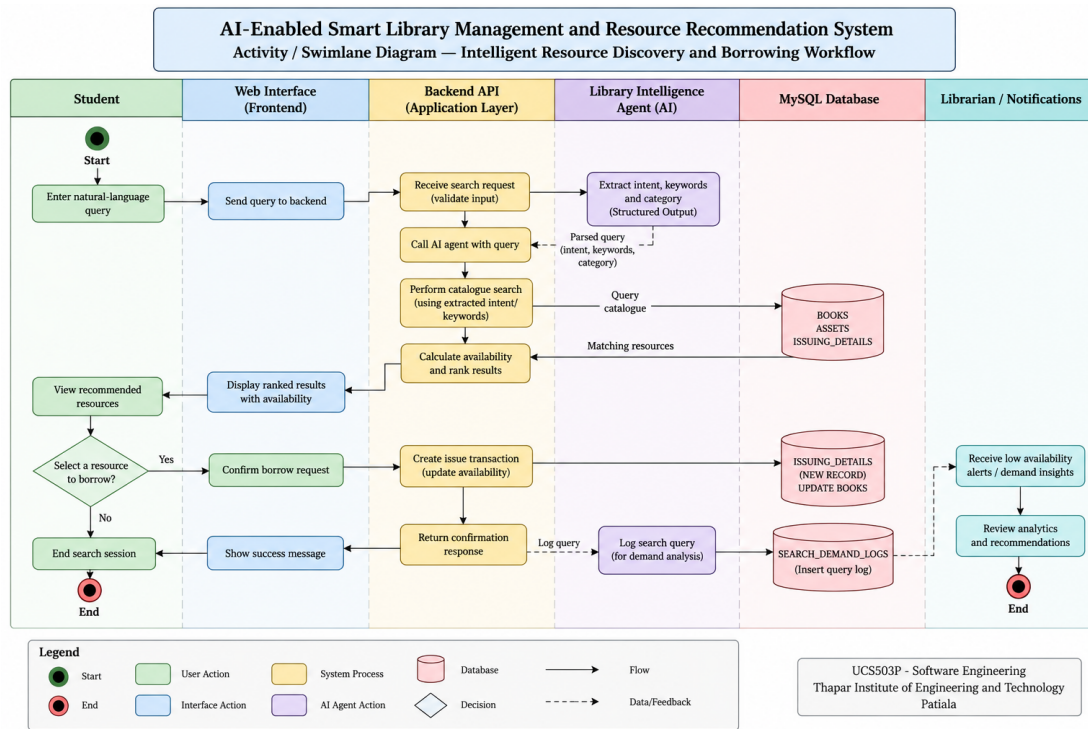


Figure 2.6: Activity and Swimlane Diagram for Intelligent Resource Discovery and Borrowing.

2.4 Database Design

The system uses MySQL as the primary relational database.

The existing core entities include student records, employee records, library assets, books, issuing transactions, and fines.

The database is extended with two additional entities:

- **NOTIFICATION_QUEUE:** Stores notifications generated for overdue and other library-management events.
- **SEARCH_DEMAND_LOGS:** Stores natural-language search queries and extracted categories for subsequent resource-demand analysis.

The database design uses primary keys, foreign-key relationships, and indexes on frequently accessed fields. This supports efficient catalogue search, transaction lookup, overdue monitoring, and demand analysis.

2.5 Library Intelligence Agent

The Library Intelligence Agent provides natural-language interaction with the library catalogue.

Its responsibilities include:

- interpreting natural-language resource requests,
- extracting relevant keywords and categories,
- searching library resources through controlled application APIs,
- checking resource availability,
- providing availability-grounded recommendations,
- answering common resource-related queries, and
- logging unfulfilled searches for subsequent demand analysis.

The agent does not receive unrestricted access to the database. Database operations are exposed through controlled application APIs and tools.

A conventional keyword-based search remains available as a fallback if the AI service is unavailable or if the extracted intent does not meet the required confidence threshold.

2.6 Record Onboarding Agent

The Record Onboarding Agent assists administrators with bulk student and member record imports.

The onboarding workflow consists of:

1. Receiving structured records.
2. Validating required fields and data types.
3. Checking roll numbers and unique identifiers.
4. Performing duplicate detection.
5. Flagging potential duplicate records for review.
6. Inserting validated records through controlled backend operations.
7. Generating an import summary.

Potential duplicate records are flagged for review rather than being automatically deleted.

2.7 Library Monitoring Agent

The Library Monitoring Agent provides scheduled monitoring and resource-management assistance.

Its responsibilities include:

- detecting resources approaching their due date,
- detecting overdue resources,
- calculating applicable overdue fines,
- generating reminder and notification records,
- analysing issue frequency and resource availability, and
- identifying high-demand and low-availability resources for purchase consideration.

Deterministic operations such as due-date comparison and fine calculation are performed by the backend and database rather than delegated to a language model.

AI functionality is used where analysis, recommendation, or summarisation provides additional value.

2.8 AI Safety and Control

The AI layer is intentionally constrained.

The agents do not receive unrestricted SQL access. Instead, database operations are exposed through predefined backend APIs and controlled tools.

The following safeguards are incorporated:

- parameterized database queries,
- role-based authorization,
- structured AI outputs,
- controlled tool invocation,
- deterministic handling of financial and circulation operations,
- backend validation before database modification, and
- conventional search fallback when AI services are unavailable.

2.9 CI/CD and Deployment

The development process incorporates continuous integration and continuous delivery practices.

The CI pipeline is intended to:

1. install project dependencies,
2. perform linting and static checks,
3. execute automated tests,
4. validate the application build, and
5. deploy successful builds to the configured environment.

This allows each development increment to be independently tested and reduces the risk of integration failures during later development stages.

Prototype Implementation and Evaluation

3.1 Prototype Implementation

The prototype establishes the major architectural components required by the project proposal.

The implementation is organized into:

- frontend web interface,
- backend REST API layer,
- MySQL database,
- controlled AI agent layer,
- automated monitoring workflow, and
- testing and deployment infrastructure.

The system is designed so that conventional library operations remain functional independently of the AI layer.

3.2 Core Library Workflow

The primary student workflow consists of:

1. Student authentication.
2. Resource search.
3. Availability verification.
4. Resource issue.
5. Due-date tracking.
6. Resource return.
7. Fine calculation where applicable.

The administrator workflow additionally provides:

1. Student/member management.
2. Resource management.
3. Bulk record onboarding.
4. Overdue monitoring.
5. Notification management.
6. Demand analysis.
7. Purchase recommendations.

3.3 Intelligent Resource Discovery Workflow

The intelligent-search workflow follows the sequence:

1. The student submits a natural-language query.
2. The Library Intelligence Agent extracts the intent, keywords, and category.
3. The backend performs controlled catalogue retrieval.
4. Resource availability is calculated using current transaction data.
5. Relevant results are ranked and returned.
6. The search is logged for subsequent demand analysis where appropriate.

The AI layer assists with interpretation and recommendation, while actual availability information is retrieved from the database.

3.4 Record Onboarding Workflow

The bulk onboarding workflow is:

1. The administrator uploads a structured batch of records.
2. The Record Onboarding Agent validates the records.
3. Duplicate and potentially duplicate records are identified.
4. Valid records are passed to the backend.
5. The backend executes insertion as a controlled transaction.
6. The system returns an import summary containing accepted and skipped records.

3.5 Monitoring and Notification Workflow

The monitoring workflow periodically evaluates active issue records.

For each active transaction:

1. The system compares the current date with the due date.
2. Overdue transactions are identified.
3. Overdue days are calculated.
4. Applicable fine records are created or updated.
5. Notification records are inserted into the notification queue.
6. Demand information is subsequently analysed using search and circulation records.

3.6 Testing Methodology

The prototype follows a layered testing approach.

3.6.1 Unit Testing

Individual backend functions and AI-agent processing components are tested independently using controlled inputs and expected outputs.

Particular attention is given to:

- input validation,
- intent extraction,
- database query functions,
- availability calculation,
- fine calculation, and
- duplicate detection.

3.6.2 Integration Testing

Integration tests verify interactions between the frontend, backend APIs, database, and controlled AI layer.

Representative workflows include:

- natural-language search to catalogue retrieval,
- resource selection to issue transaction,
- bulk onboarding to database insertion, and
- overdue detection to notification generation.

3.6.3 Performance Testing

Performance testing measures API response times and resource-discovery latency under controlled test conditions.

The database is evaluated for common catalogue and circulation queries, with indexing and pagination used to support response-time requirements.

3.6.4 Functional Testing

Functional testing verifies that the principal student and administrator workflows behave according to their expected requirements.

3.7 Evaluation Criteria

The prototype is evaluated using measurable workflow, accuracy, performance, and reliability criteria.

3.7.1 Primary Metrics

Table 3.1: Primary Prototype Evaluation Metrics

Metric	Measurement	Target
Resource discovery time	Time required to locate a requested or suitable available resource for predefined queries	≤ 10 seconds
Duplicate detection precision	Proportion of records flagged as duplicates that are actual duplicates in a labelled dataset	$\geq 90\%$
Overdue detection coverage	Proportion of known overdue transactions correctly detected	$\geq 99\%$
Demand recommendation accuracy	Proportion of high-demand and low-availability resources correctly identified according to predefined thresholds	$\geq 90\%$

3.7.2 Secondary Metrics

Table 3.2: Secondary Prototype Evaluation Metrics

Metric	Measurement	Target
API response time	Median response time for common non-AI application requests	≤ 500 ms
AI answer validity	Proportion of predefined library questions answered consistently with database information	$\geq 90\%$
Notification delay	Time between an overdue condition becoming eligible and generation of its notification	Measured experimentally
System reliability	Availability of the deployed application during the evaluation period	$\geq 99\%$
Test coverage	Percentage of critical backend logic covered by automated tests	$\geq 70\%$

3.8 Validation Strategy

The evaluation uses controlled datasets and predefined scenarios.

For resource discovery, conventional search can be compared with natural-language interaction by measuring the time required to locate resources.

For onboarding, a labelled dataset containing valid, invalid, and duplicate records can be used to evaluate duplicate-detection precision.

For monitoring, synthetic borrowing records with known due dates and overdue states can be used to measure detection coverage and notification latency.

For demand analysis, historical or synthetic issue records can be used to define high-demand resources according to predetermined thresholds. The resulting recommendations can then be compared against expected labels.

3.9 Analysis

The prototype is evaluated primarily on functional correctness, integration, reliability, and measurable performance rather than on novelty of the underlying AI models.

The architecture provides a clear separation between deterministic operations and AI-assisted operations. This reduces the risk of hallucinated or inconsistent information affecting critical library transactions.

The principal engineering challenge is integrating AI-assisted features without making the core system dependent on an external AI service. Maintaining conventional search and deterministic backend workflows as fallbacks addresses this concern.

The evaluation targets provide a basis for measuring whether the prototype meets the intended software-engineering objectives. Actual achieved values should be recorded after executing the corresponding controlled test datasets and performance experiments.

Scalability and Deployment

4.1 Scalability

The system is designed to scale in both theoretical and practical terms.

4.1.1 Logical Scalability

- Frontend and backend responsibilities are separated.
- Backend APIs are designed to be stateless where practical.
- Database queries use indexing and pagination for frequently accessed records.
- AI agents are modular and can be extended or replaced without rewriting the complete application.

4.1.2 Operational Deployability

- The application can be containerized using Docker.
- A managed relational database can be used for deployment.
- CI/CD supports repeatable staging and production deployments.
- The architecture can support a larger number of users and records without requiring a fundamental redesign.

4.2 Engine Availability

The project relies on mature and readily available software engines rather than requiring fundamental algorithmic research.

The required engineering components include:

- a modern web framework for frontend and backend development,
- REST API infrastructure for application integration,
- MySQL or another managed relational database,
- authentication and role-based authorization libraries,

- mature LLM or AI APIs and supporting agent frameworks,
- automated testing frameworks,
- GitHub Actions or an equivalent CI runner, and
- Docker and a platform-hosted or cloud deployment target.

4.3 Deployment Strategy

The deployment strategy follows the project's incremental development approach.

The application can first be deployed to a staging environment for validation. Successful builds can subsequently be promoted to the production environment through the CI/CD pipeline.

The separation between frontend, backend, database, and AI components allows individual services to be scaled according to workload.

4.4 Risks and Mitigations

- **AI hallucination or incorrect resource information:** AI responses are grounded in current database records through controlled retrieval tools and predefined APIs.
- **Excessive project scope:** The core library workflow is completed before AI features are added, and later functionality is delivered incrementally.
- **Duplicate detection errors:** Potential duplicates are flagged for human review instead of being automatically deleted.
- **AI service unavailability:** Conventional keyword-based resource search remains available as a fallback.
- **Unauthorized automated actions:** Agents operate through role-controlled APIs, and irreversible operations require explicit backend authorization.
- **Insufficient evaluation data:** Synthetic and labelled library datasets can be prepared so that evaluation does not depend entirely on live institutional data.
- **Deployment instability:** A staging environment and CI/CD pipeline make deployments repeatable.
- **Database performance degradation:** Frequently accessed fields are indexed and database operations use pagination and optimized queries.

Conclusion

5.1 Summary of Work

The prototype establishes the foundation of an AI-enabled Smart Library Management System that combines conventional library workflows with controlled AI-assisted functionality.

The system addresses four principal areas:

1. resource discovery,
2. student/member record onboarding,
3. overdue and notification management, and
4. resource demand analysis.

The architecture separates the frontend, backend, database, and AI agent layers, allowing individual components to be developed and tested independently.

5.2 Key Findings

The prototype demonstrates how AI can be incorporated into a conventional software system without allowing the AI layer to directly control critical database operations.

The appropriate separation of responsibilities is:

- **Deterministic backend/database operations:** availability calculation, issue and return transactions, due-date comparison, and fine calculation.
- **AI-assisted operations:** natural-language interpretation, resource recommendation, duplicate-detection assistance, demand analysis, and summarisation.

This separation improves reliability while still providing useful AI-assisted interaction.

5.3 Future Work and Recommendations

Future development can extend the prototype through:

1. improved semantic and vector-based resource search,

2. more advanced recommendation ranking,
3. improved duplicate-detection and entity-resolution methods,
4. richer librarian analytics,
5. improved demand forecasting,
6. automated notification delivery,
7. support for additional resource types beyond books,
8. stronger authentication and role-management mechanisms, and
9. deployment under a production cloud environment.

5.4 Final Remarks

The project follows the software-engineering objective of delivering practical value through incremental development rather than attempting to solve an open-ended AI research problem.

The prototype provides a foundation for a deployable library-management platform in which conventional software handles critical deterministic operations and controlled AI agents provide natural-language interaction, automation, and decision support.

The system can therefore be evaluated using measurable criteria covering functionality, accuracy, performance, reliability, and scalability.